

Filament Forms — Practical Notes

Everything you need to build a flowing, reactive Filament web app: every common form field, the modifiers that matter, layout components, validation, and the reactive plumbing (`live()`, `$get`/`$set`, `afterStateUpdated`) that makes forms feel alive.

Notes target Filament v5 (current: v5.6.3, May 2026). Most of the form API is identical across v3 → v5; differences are flagged inline.

1. What Filament Is (and Where Forms Fit)

Filament is a **server-driven UI toolkit for Laravel**, built on Livewire + Alpine + Tailwind. You describe what you want in PHP — fields, columns, actions — and Filament renders the HTML, wires up the reactivity, and handles the round trips for you. No Vue, no React, no manual AJAX.

Filament ships as a set of composable packages:

Package	What it does
<code>filament/forms</code>	Reactive form builder — the focus of these notes
<code>filament/tables</code>	Data tables with filters, search, actions
<code>filament/infolists</code>	Read-only "detail view" layouts
<code>filament/notifications</code>	Toast / database notifications
<code>filament/actions</code>	Buttons that trigger modals/forms/confirmations
<code>filament/widgets</code>	Dashboard widgets (charts, stats)
<code>filament/schemas</code>	The v4+ "glue" — unifies form/infolist/layout components
<code>filament/filament</code>	The full admin panel that ties it all together

You can use the form builder **standalone** in a plain Livewire component, or inside the full **Panel Builder** (`/admin` style apps). Either way, the form API is identical — that's the whole point.

NOTE

Since Filament v4, every UI component (form field, infolist entry, layout block) lives under the unified **Schema** concept. In practice this means you can mix and match — drop a `TextEntry` into a form, or a `Section` into an infolist. The form builder remains the most-used entry point.

2. The Mental Model

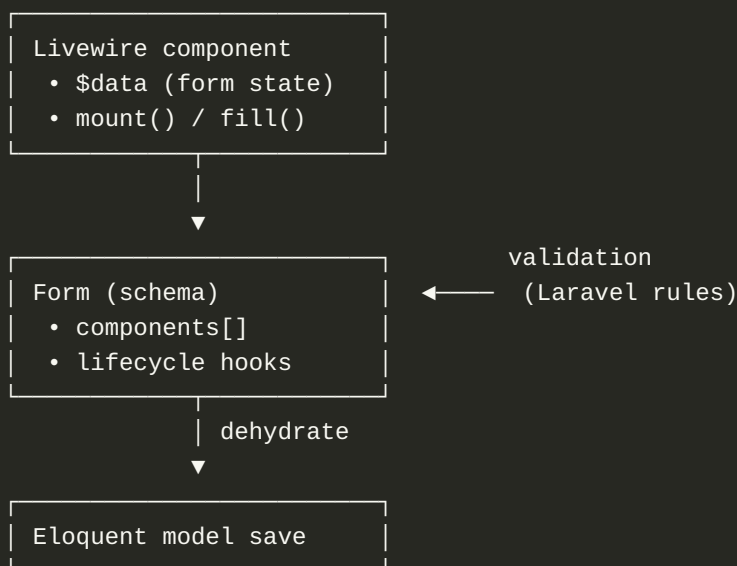
A form is a schema

A Filament form is just an **array of components**, passed to the `schema()` method of a `Form` (or `Schema` in v5). Each component is a configuration object — you build it with static method chaining:

```
TextInput::make('email')
    ->email()
    ->required()
    ->maxLength(255);
```

That single chain produces a rendered `<input type="email">` with validation, a label, and Livewire reactivity. You never touch Blade for the input itself.

The data flow



Two terms you'll see constantly:

- **Hydrate** — Filament reads a model and pours its attributes into the form fields (when editing).
- **Dehydrate** — Filament reads the form state back out and prepares it for saving. Skipping dehydration (`->dehydrated(false)`) means "don't include this value in the saved payload."

Where a form can live

Forms aren't only inside admin Resources. You'll meet them in four places:

1. **Resource pages** — `CreateRecord`, `EditRecord`. The `form()` method on the Resource defines the schema. This is where 90% of admin forms live.
2. **Custom Livewire components** — drop the `InteractsWithForms` trait, define `form()`, render `{{ $this->form }}`.
3. **Action modals** — `Action::make('foo')->form([...])` opens a modal with a form. Great for one-off operations (assign user, send email, etc.).
4. **Table filters & bulk actions** — use the same field components inside filter schemas.

The schema you write doesn't change between contexts.

3. The Anatomy of a Form Field

Every field shares a common base. Learn this once and the rest is just memorising components.

```
TextInput::make('column_name')           // maps to DB column / array key
->label('Display Label')                  // override auto-generated label
->helperText('Shown below the field')     // small grey hint
->placeholder('e.g. Acme Corp')           // input placeholder
->default('initial value')                 // default for create form
->required()                             // validation
->disabled()                              // read-only on screen
->dehydrated(false)                       // never save to DB
->visible(fn () => true)                   // conditional render
->hidden(fn () => false)                   // inverse of visible
->columnSpan(2)                           // grid span (1, 2, 'full')
->columnSpanFull()                        // shortcut for full width
->live()                                  // re-render on change
->afterStateUpdated(function () {...})    // hook after value changes
->extraAttributes(['class' => 'bg-red-50']); // raw HTML attrs
```

These ~15 modifiers cover most of what you'll write day-to-day. The field-specific methods (e.g. `->email()`, `->numeric()`, `->options()`) layer on top.

GOTCHA

Modifier methods are ****chained, not nested****. Forgetting a semicolon at the end of a chain breaks PHP, not Filament. If a field "isn't working" with a clean error, check that you didn't put a ``;` between two ``->`` calls.

4. The Field Reference

Each subsection below has: import, minimal example, the modifiers that matter, and a "when to use it" note.

4.1 `TextInput` — the workhorse

```
use Filament\Forms\Components\TextInput;

TextInput::make('name')
    ->required()
    ->maxLength(255)
    ->minLength(2);
```

Common type shortcuts (each sets `type=` on the underlying `<input>`):

```
TextInput::make('email')->email();           // type=email + validation
TextInput::make('password')->password();       // type=password
TextInput::make('age')->numeric();              // type=number, integer cast
TextInput::make('price')->numeric()->step(0.01); // decimals
TextInput::make('phone')->tel();                // type=tel
TextInput::make('website')->url();              // type=url + validation
```

Affixes — text/icons on either side of the field:

```
TextInput::make('price')
    ->numeric()
    ->prefix('R')                // currency symbol before
    ->suffix('.00')               // text after
    ->prefixIcon('heroicon-m-currency-dollar')
    ->suffixIcon('heroicon-m-check-circle');
```

Password reveal + confirmation pattern:

```
TextInput::make('password')
    ->password()
    ->revealable()                // eye icon to toggle visibility
    ->required()
    ->confirmed();                // requires matching password_confirmation
    field

TextInput::make('password_confirmation')
    ->password()
    ->revealable()
    ->dehydrated(false);          // never save this field
```

Input masks (requires `->mask()`, takes an Alpine `$input` expression):

```
TextInput::make('card_number')
    ->mask('9999 9999 9999 9999');
```

4.2 Textarea — multi-line text

```
use Filament\Forms\Components\Textarea;

Textarea::make('bio')
    ->rows(5)
    ->cols(40)
    ->maxLength(1000)
    ->autosize(); // grows as user types
```

Use for short paragraphs, internal notes, plain-text descriptions. For anything formatted (bold, links, lists), use `RichEditor` or `MarkdownEditor`.

4.3 Select — single or multi-select dropdown

The most feature-loaded field in the library. Three flavours: static options, dynamic options, Eloquent relationship.

Static options:

```
use Filament\Forms\Components\Select;

Select::make('status')
    ->options([
        'draft' => 'Draft',
        'reviewing' => 'Reviewing',
        'published' => 'Published',
    ])
    ->default('draft')
    ->required();
```

Searchable + multiple:

```
Select::make('tags')
    ->multiple()
    ->options(Tag::pluck('name', 'id'))
    ->searchable()
    ->preload(); // load all options up front
```

Eloquent relationship (the killer feature):

```
Select::make('user_id')
    ->relationship('user', 'name')           // belongsTo: $post->user
    ->searchable()
    ->preload()
    ->createOptionForm([                     // "+ Create new" modal
        TextInput::make('name')->required(),
        TextInput::make('email')->email()->required(),
    ]);
```

This auto-handles the `belongsTo` write-back, plus opens a modal to create a related record inline. For `BelongsToMany` use the same `->relationship()` with `->multiple()`.

Dynamic options based on another field:

```
Select::make('country_id')
    ->options(Country::pluck('name', 'id'))
    ->live();                               // re-render dependent fields on change

Select::make('state_id')
    ->options(fn (Get $get) =>
        State::where('country_id', $get('country_id'))->pluck('name', 'id')
    );
```

NOTE ``Get $get`` is a callable utility Filament injects into closures. You can also write it as ``function ($get) {...}`` — Filament resolves it by parameter name. Same goes for ``$set``, ``$state``, ``$record``, ``$operation``, ``$livewire``.

4.4 `Checkbox` and `Toggle` — booleans

```
use Filament\Forms\Components\Checkbox;
use Filament\Forms\Components\Toggle;

Checkbox::make('terms_accepted')
    ->required()
    ->accepted();                           // must be true

Toggle::make('is_active')
    ->onIcon('heroicon-m-check')
    ->offIcon('heroicon-m-x-mark')
    ->onColor('success')
    ->offColor('danger');
```

`Toggle` is a styled checkbox — use it for on/off settings, use `Checkbox` for acknowledgements / consents.

4.5 `CheckboxList` and `Radio` — multiple choice

```
use Filament\Forms\Components\CheckboxList;
use Filament\Forms\Components\Radio;

CheckboxList::make('permissions')
    ->options([
        'create' => 'Create posts',
        'edit'    => 'Edit posts',
        'delete'  => 'Delete posts',
    ])
    ->columns(2) // lay out in 2 columns
    ->bulkToggleable(); // "select all" toggle

Radio::make('priority')
    ->options([
        'low' => 'Low',
        'med' => 'Medium',
        'high' => 'High',
    ])
    ->descriptions([ // small text under each option
        'low' => 'Whenever you get to it',
        'high' => 'Drop everything',
    ])
    ->inline(); // horizontal layout
```

4.6 `DatePicker` / `DateTimePicker` / `TimePicker`

```
use Filament\Forms\Components\DatePicker;
use Filament\Forms\Components\DateTimePicker;

DatePicker::make('date_of_birth')
    ->maxDate(now()) // can't be in the future
    ->minDate(now()->subYears(120))
    ->displayFormat('d/m/Y') // user-facing format
    ->format('Y-m-d') // DB format
    ->native(false); // use JS picker, not browser native

DateTimePicker::make('published_at')
    ->seconds(false) // hide seconds input
    ->minutesStep(15) // 15-min increments
    ->timezone('Africa/Johannesburg');
```


4.7 **FileUpload** — single or multiple files

```
use Filament\Forms\Components\FileUpload;

FileUpload::make('avatar')
    ->image() // limit MIME to images
    ->disk('public') // storage disk (config/filesystems.php)
    ->directory('avatars') // subfolder
    ->visibility('public')
    ->maxSize(2048) // KB → 2MB
    ->imageEditor() // built-in crop/rotate UI
    ->imageAspectRatio('1:1');
```

Multiple files + reordering:

```
FileUpload::make('attachments')
    ->multiple()
    ->reorderable()
    ->maxFiles(5)
    ->acceptedFileTypes(['application/pdf'])
    ->downloadable() // show download button on edit
    ->previewable();
```

WARNING

`FileUpload` saves files **immediately** on selection (it's a Livewire temporary upload that gets moved on form submit). If a user picks a file and abandons the form, the temp file lingers until Laravel's cleanup runs. Don't be surprised by orphan files in `livewire-tmp`.

4.8 **RichEditor** — TipTap-based WYSIWYG

```
use Filament\Forms\Components\RichEditor;

RichEditor::make('content')
    ->toolbarButtons([
        'bold', 'italic', 'link', 'bulletList', 'orderedList',
        'h2', 'h3', 'blockquote', 'codeBlock',
    ])
    ->fileAttachmentsDisk('public')
    ->fileAttachmentsDirectory('attachments')
    ->columnSpanFull();
```

Stores HTML. For blog posts, product descriptions, anything where editors need formatting.

4.9 **MarkdownEditor** — split-pane markdown

Same shape as **RichEditor** but stores raw markdown. Use when you want portable, version-controllable content (docs, READMEs, knowledge base).

```
use Filament\Forms\Components\MarkdownEditor;

MarkdownEditor::make('readme')
    ->toolbarButtons(['bold', 'italic', 'link', 'codeBlock', 'preview']);
```

4.10 **ColorPicker**

```
use Filament\Forms\Components\ColorPicker;

ColorPicker::make('brand_color')
    ->hex(); // hex(), rgb(), rgba(), hsl()
```

4.11 **TagsInput** — comma-separated values

```
use Filament\Forms\Components\TagsInput;

TagsInput::make('keywords')
    ->separator(',')
    ->suggestions(['laravel', 'php', 'filament']);
```

Saves as JSON array by default. Cast the model attribute as **array** to read it back.

4.12 **KeyValue** — arbitrary key/value pairs

```
use Filament\Forms\Components\KeyValue;

KeyValue::make('metadata')
    ->keyLabel('Property')
    ->valueLabel('Value')
    ->reorderable()
    ->addActionLabel('Add property');
```

Use for sparse, schema-less data (HTTP headers, env-like config, feature flags on a record).

4.13 **Hidden** — invisible field

```
use Filament\Forms\Components\Hidden;

Hidden::make('user_id')
    ->default(fn () => auth()->id());
```

For values you set programmatically but still want included in the form state.

4.14 **ToggleButtons** — pretty radio/checkbox alternative

```
use Filament\Forms\Components\ToggleButtons;

ToggleButtons::make('priority')
    ->options([
        'low' => 'Low',
        'med' => 'Medium',
        'high' => 'High',
    ])
    ->colors([
        'low' => 'gray',
        'med' => 'warning',
        'high' => 'danger',
    ])
    ->icons([
        'low' => 'heroicon-m-arrow-down',
        'high' => 'heroicon-m-arrow-up',
    ])
    ->inline();
```

Visually nicer than **Radio** for short lists. Supports **->multiple()** to behave like checkboxes.

4.15 **Slider** (v4+)

```
use Filament\Forms\Components\Slider;

Slider::make('rating')
    ->minValue(0)
    ->maxValue(10)
    ->step(0.5);
```

Range version (two handles):

```
Slider::make('price_range')
    ->range()
    ->minValue(0)
    ->maxValue(1000);
```

4.16 **CodeEditor** (v4+)

```
use Filament\Forms\Components\CodeEditor;

CodeEditor::make('snippet')
    ->language('php') // html, css, js, php, json
    ->columnSpanFull();
```

Syntax-highlighted multiline editor. Saves the raw string.

4.17 **Repeater** — dynamic array of sub-forms

The single most important field for "flowing" interactive forms. Lets users add/remove rows of structured data.

```
use Filament\Forms\Components\Repeater;

Repeater::make('line_items')
    ->schema([
        TextInput::make('description')->required(),
        TextInput::make('quantity')->numeric()->default(1),
        TextInput::make('unit_price')->numeric()->prefix('R'),
    ])
    ->columns(3)
    ->defaultItems(1)
    ->minItems(1)
    ->maxItems(20)
    ->reorderable()
    ->collapsible()
    ->itemLabel(fn (array $state): ?string => $state['description'] ?? null)
    ->addActionLabel('Add line item');
```

For a true one-to-many DB relationship:

```
Repeater::make('addresses')
    ->relationship() // auto-handles the hasMany
    ->schema([
        TextInput::make('street')->required(),
        TextInput::make('city')->required(),
    ]);
```

GOTCHA

A ``Repeater`` without ``->relationship()`` saves to a JSON column. Make sure your model casts the attribute as ``array``, or the data round-trips wrong on edit.

4.18 **Builder** — heterogeneous repeater

Like **Repeater** but each row can be one of several "block" types. Perfect for page builders / CMS content.

```
use Filament\Forms\Components\Builder;
use Filament\Forms\Components\Builder\Block;

Builder::make('content')
    ->blocks([
        Block::make('heading')
            ->schema([
                TextInput::make('text')->required(),
                Select::make('level')->options(['h1' => 'H1', 'h2' => 'H2']),
            ]),

        Block::make('paragraph')
            ->schema([
                Textarea::make('text')->required(),
            ]),

        Block::make('image')
            ->schema([
                FileUpload::make('url')->image()->required(),
                TextInput::make('alt'),
            ]),
    ])
    ->collapsible();
```

4.19 **Placeholder** — static text inside a form

Not a field, not editable — a labelled value to display:

```
use Filament\Forms\Components\Placeholder;

Placeholder::make('created_at')
    ->label('Created')
    ->content(fn ($record) => $record?->created_at?->diffForHumans() ?? 'New');
```

Use to surface computed/read-only context inside a form (timestamps, computed totals, status badges).

5. Layout Components

Fields are content. Layout components are scaffolding. They take a `schema()` of children and arrange them.

5.1 `Grid` — multi-column rows

```
use Filament\Forms\Components\Grid;

Grid::make(3)
    ->schema([
        TextInput::make('first_name'),
        TextInput::make('last_name'),
        TextInput::make('email')->columnSpan(2),
    ]);
```

Responsive columns:

```
Grid::make([
    'default' => 1,
    'sm' => 2,
    'lg' => 3,
])->schema([ /* ... */ ]);
```

5.2 `Section` — bordered card with heading

```
use Filament\Forms\Components\Section;

Section::make('Contact Details')
    ->description('How customers reach you')
    ->icon('heroicon-m-phone')
    ->schema([
        TextInput::make('phone'),
        TextInput::make('email'),
    ])
    ->columns(2)
    ->collapsible()
    ->collapsed(); // start collapsed
```

Use to break long forms into logical chunks.

5.3 `Fieldset` — lightweight grouping

Like `Section` but no card chrome — just a bordered label.

```
use Filament\Forms\Components\Fieldset;

Fieldset::make('Address')
    ->schema([
        TextInput::make('street')->columnSpanFull(),
        TextInput::make('city'),
        TextInput::make('postal_code'),
    ]);
```

5.4 **Tabs** — sectioned navigation

```
use Filament\Forms\Components\Tabs;
use Filament\Forms\Components\Tabs\Tab;

Tabs::make('Settings')
    ->tabs([
        Tab::make('General')->schema([ /* ... */ ]),
        Tab::make('Billing')->icon('heroicon-m-credit-card')->schema([ /* ... */ ]),
        Tab::make('Notifications')->badge('3')->schema([ /* ... */ ]),
    ])
    ->columnSpanFull();
```

5.5 **Wizard** — multi-step form

Best UX for long forms that would otherwise scroll forever.

```

use Filament\Forms\Components\Wizard;
use Filament\Forms\Components\Wizard\Step;

Wizard::make([
    Step::make('Account')
        ->description('Login credentials')
        ->schema([
            TextInput::make('email')->email()->required(),
            TextInput::make('password')->password()->required(),
        ]),

    Step::make('Profile')
        ->schema([
            TextInput::make('name')->required(),
            FileUpload::make('avatar')->image(),
        ]),

    Step::make('Preferences')
        ->schema([
            Toggle::make('notifications'),
            Select::make('timezone')->options(/* ... */),
        ]),
])
->submitAction(new HtmlString(Blade::render('<x-filament::button type="submit">Sign
up</x-filament::button>'))))
->columnSpanFull();

```

By default, wizards validate **per step** before letting the user advance.

5.6 **split** — side-by-side panels

```

use Filament\Forms\Components\Split;

Split::make([
    Section::make('Content')->schema([ /* ... */ ]),
    Section::make('Meta')->grow(false)->schema([ /* ... */ ]),
])->columnSpanFull();

```

`->grow(false)` makes a panel take only what it needs (sidebar pattern).

6. Validation

Filament wraps Laravel's validator. You have three ways to attach rules:

Method 1: Built-in fluent helpers (preferred):


```

TextInput::make('email')
    ->required()
    ->email()
    ->unique(ignoreRecord: true)           // unique on the model's table
    ->maxLength(255);

TextInput::make('age')
    ->numeric()
    ->minValue(0)
    ->maxValue(150);

TextInput::make('username')
    ->alphaDash()
    ->minLength(3)
    ->maxLength(20);

```

Most Laravel rules have a Filament shortcut: `->required()`, `->nullable()`, `->email()`, `->url()`, `->numeric()`, `->integer()`, `->alpha()`, `->alphaDash()`, `->alphaNum()`, `->minLength()`, `->maxLength()`, `->minValue()`, `->maxValue()`, `->regex()`, `->same('other_field')`, `->different('other_field')`, `->confirmed()`, `->accepted()`, `->in([...])`, `->notIn([...])`, `->exists('table', 'column')`, `->unique()`.

Method 2: Raw Laravel rules array:

```

TextInput::make('vat_number')
    ->rules(['required', 'string', 'starts_with:ZA'])
    ->rule(new ValidVatNumber);           // custom Rule instance

```

Method 3: Conditional rules:

```

TextInput::make('company_name')
    ->required(fn (Get $get) => $get('is_business') === true);

```

Almost every modifier accepts either a value or a closure — the closure is re-evaluated when reactive state changes.

Custom messages:

```

TextInput::make('email')
    ->required()
    ->email()
    ->validationMessages([
        'required' => 'We need your email to send confirmations.',
        'email'    => 'That doesn\'t look like a valid email address.',
    ]);

```

7. Reactive Forms — The Flow

This is where Filament earns its money. Three primitives make forms feel alive:

7.1 `live()` — re-render on change

By default, Filament only updates the form state on submit. `->live()` tells Livewire to push the field's value to the server on every change, triggering a re-render that lets dependent fields react.

```
Checkbox::make('is_business')->live();

TextInput::make('vat_number')
    ->visible(fn (Get $get) => $get('is_business'));
```

For text inputs you usually want debouncing to avoid hammering the server:

```
TextInput::make('search')
    ->live(debounce: 500);                // ms

TextInput::make('slug')
    ->live(onBlur: true);                // fire only on blur
```

7.2 `afterStateUpdated()` — hooks

Run code when a field's value changes:

```
TextInput::make('title')
    ->live(onBlur: true)
    ->afterStateUpdated(function (Set $set, $state) {
        $set('slug', Str::slug($state));
    });

TextInput::make('slug')->required();
```

Typing a title now auto-fills the slug on blur. `Set $set` is another injected utility — it writes to other fields by name.

7.3 `$get` / `$set` — read and write other fields

Used inside closures:

```

Select::make('country_id')
  ->options(Country::pluck('name', 'id'))
  ->live()
  ->afterStateUpdated(fn (Set $set) => $set('state_id', null));

Select::make('state_id')
  ->options(fn (Get $get) =>
    State::where('country_id', $get('country_id'))->pluck('name', 'id')
  );

```

Classic dependent-dropdown pattern: clear the dependent field when the parent changes, then dynamically fetch its options.

7.4 Conditional visibility / disabling

```

TextInput::make('shipping_address')
  ->visible(fn (Get $get) => $get('needs_shipping'))
  ->required(fn (Get $get) => $get('needs_shipping'));

DatePicker::make('cancelled_at')
  ->disabled(fn ($record) => $record?->is_locked);

```

`visible()` removes the field from the DOM (and from validation). `hidden()` is the inverse. `disabled()` keeps it rendered but read-only. `readOnly()` is the lighter touch — user can't type, but can copy.

7.5 `jsAction()` (v4.5+) — client-side actions

For trivial state updates, you can skip the network round trip entirely:

```

use Filament\Actions\Action;

TextInput::make('title')
  ->live(onBlur: true)
  ->suffixAction(
    Action::make('generateSlug')
      ->icon('heroicon-m-sparkles')
      ->jsAction(<<<'JS'
        $set('slug', $get('title').toLowerCase().replaceAll(' ', '-'))
      JS)
  );

TextInput::make('slug');

```

NOTE

``jsAction()`` runs in the browser via Alpine — instant, no server hit. Use it for cheap UI tricks like copying state between fields. Anything that touches the DB or external APIs still needs a normal action.

8. Relationships in Practice

8.1 `belongsTo` — single Select

```
Select::make('user_id')
    ->relationship('user', 'name')           // method name on model, display column
    ->searchable()
    ->preload();
```

8.2 `belongsToMany` — multi-select

```
Select::make('tags')
    ->relationship('tags', 'name')
    ->multiple()
    ->preload();
```

Filament handles the sync automatically.

8.3 `hasMany` — repeater with `->relationship()`

```
Repeater::make('items')
    ->relationship()                         // method must exist on model
    ->schema([
        TextInput::make('name')->required(),
        TextInput::make('quantity')->numeric(),
    ]);
```

8.4 Inline create — `createOptionForm`

```
Select::make('category_id')
  ->relationship('category', 'name')
  ->createOptionForm([
    TextInput::make('name')->required(),
    TextInput::make('description'),
  ])
  ->editOptionForm([ // pencil icon to edit
    TextInput::make('name')->required(),
  ]);
```

User clicks "+ Create", a modal opens with the form, the new option is saved and selected — no leaving the parent form.

8.5 Filtering options dynamically

```
Select::make('user_id')
  ->relationship(
    name: 'user',
    titleAttribute: 'name',
    modifyQueryUsing: fn ($query) =>
      $query->where('is_active', true)->orderBy('name'),
  );
```

9. Form Hooks & Lifecycle

When forms live in `CreateRecord` / `EditRecord` pages, you get lifecycle hooks for the surrounding save flow:

```

class CreatePost extends CreateRecord
{
    protected static string $resource = PostResource::class;

    protected function mutateFormDataBeforeCreate(array $data): array
    {
        // Modify form payload before save
        $data['user_id'] = auth()->id();
        $data['slug'] = Str::slug($data['title']);
        return $data;
    }

    protected function afterCreate(): void
    {
        // Side effects after save
        $this->record->notifySubscribers();
    }

    protected function getRedirectUrl(): string
    {
        return $this->getResource()->getUrl('index');
    }
}

```

Equivalent hooks on `EditRecord`: `mutateFormDataBeforeFill()`, `mutateFormDataBeforeSave()`, `afterSave()`.

For field-level hooks, the chain is: `dehydrateStateUsing()`, `mutateDehydratedStateUsing()`, `mutateRelationshipDataBeforeCreateUsing()`. Use these when you need to transform a single field's value on its way to/from the DB (e.g., hashing passwords, encrypting secrets).

```

TextInput::make('password')
    ->password()
    ->dehydrateStateUsing(fn ($state) => Hash::make($state))
    ->dehydrated(fn ($state) => filled($state)) // skip when blank on edit
    ->required(fn (string $operation) => $operation === 'create');

```

10. Putting It Together — A Complete Resource

A `Product` resource showing most of the patterns above:

```

namespace App\Filament\Resources;

use App\Filament\Resources\ProductResource\Pages;
use App\Models\Product;
use Filament\Forms;
use Filament\Forms\Components\{
    Section, Grid, TextInput, Textarea, Select, FileUpload,
    Toggle, RichEditor, Repeater, DatePicker
};
use Filament\Forms\Form;
use Filament\Forms\Get;
use Filament\Forms\Set;
use Filament\Resources\Resource;
use Illuminate\Support\Str;

class ProductResource extends Resource
{
    protected static ?string $model = Product::class;
    protected static ?string $navigationIcon = 'heroicon-o-shopping-bag';

    public static function form(Form $form): Form
    {
        return $form->schema([

            Section::make('Basic Info')
                ->columns(2)
                ->schema([

                    TextInput::make('name')
                        ->required()
                        ->maxLength(255)
                        ->live(onBlur: true)
                        ->afterStateUpdated(fn (Set $set, $state) =>
                            $set('slug', Str::slug($state))
                        ),

                    TextInput::make('slug')
                        ->required()
                        ->unique(ignoreRecord: true)
                        ->maxLength(255),

                    Select::make('category_id')
                        ->relationship('category', 'name')
                        ->searchable()
                        ->preload()
                        ->required()
                        ->createOptionForm([
                            TextInput::make('name')->required(),
                        ]),

                    Select::make('tags')
                        ->relationship('tags', 'name')
                        ->multiple()
                        ->preload(),

                ]),

        ]),
    }
}

```

```

Section::make('Description')
  ->schema([
    Textarea::make('short_description')
      ->maxLength(500)
      ->rows(3),

    RichEditor::make('description')
      ->columnSpanFull(),
  ]),

Section::make('Pricing & Stock')
  ->columns(3)
  ->schema([
    TextInput::make('price')
      ->numeric()
      ->prefix('R')
      ->minValue(0)
      ->required(),

    TextInput::make('compare_at_price')
      ->numeric()
      ->prefix('R')
      ->helperText('Original price, shown struck through'),

    TextInput::make('stock')
      ->numeric()
      ->minValue(0)
      ->default(0),
  ]),

Section::make('Variants')
  ->schema([
    Repeater::make('variants')
      ->relationship()
      ->schema([
        TextInput::make('name')->required(),
        TextInput::make('sku')->required(),
        TextInput::make('stock')->numeric()->default(0),
      ])
      ->columns(3)
      ->reorderable()
      ->collapsible()
      ->itemLabel(fn (array $state) => $state['name'] ?? 'New
variant'),
  ]),

Section::make('Media')
  ->schema([
    FileUpload::make('images')
      ->image()
      ->multiple()
      ->reorderable()
      ->maxFiles(10)
  ])

```



```

                ->directory('products')
                ->columnSpanFull(),
            )),
        Section::make('Publishing')
            ->columns(2)
            ->schema([
                Toggle::make('is_published')
                    ->live(),

                DatePicker::make('published_at')
                    ->visible(fn (Get $get) => $get('is_published'))
                    ->default(now()),
            ]),
    ]);
}

// tables(), getPages() omitted for brevity
}

```

What this gives you, out of the box:

- Title → slug auto-population
- Inline category creation
- Multi-tag selection with a pivot table
- Rich-text product description
- Numeric pricing with currency prefix
- Variant repeater with reorder/collapse
- Multi-image upload with drag-to-reorder
- Conditional "Published at" date that only appears when "Is published" is on

No JavaScript written. No Blade touched. ~120 lines of PHP for a feature that would be a week of work without Filament.

11. Common Patterns

11.1 Don't save a field to DB

```

TextInput::make('password_confirmation')
    ->password()
    ->dehydrated(false);

```

`dehydrated(false)` means "include in the form for the user, but never include in the save payload."

11.2 Different requirements for create vs edit

```
TextInput::make('password')
    ->password()
    ->required(fn (string $operation) => $operation === 'create')
    ->dehydrated(fn ($state) => filled($state))
    ->dehydrateStateUsing(fn ($state) => Hash::make($state));
```

`$operation` is injected; values are `'create'`, `'edit'`, or `'view'`.

11.3 Computed default values

```
TextInput::make('reference')
    ->default(fn () => 'INV-' . str_pad(Invoice::max('id') + 1, 6, '0',
STR_PAD_LEFT));
```

11.4 Show form context to the user

```
Placeholder::make('total')
    ->label('Order total')
    ->content(fn (Get $get) =>
        'R ' . number_format(collect($get('line_items')) ?? [])
            ->sum(fn ($i) => ($i['quantity'] ?? 0) * ($i['unit_price'] ?? 0)), 2)
    );
```

Combined with `live()` on the repeater fields, this gives a live order total.

11.5 Wizard with validation gating

```
Wizard::make([
    Step::make('Account')->schema([
        TextInput::make('email')->email()->required(),
    ]),
    Step::make('Profile')->schema([
        TextInput::make('name')->required(),
    ]),
])
->skippable() // allow jumping between steps
->persistStepInQueryString(); // step in URL for shareable state
```

11.6 Standalone form in a Livewire component

When you're not in a Resource:

```

namespace App\Livewire;

use Filament\Forms\Concerns\InteractsWithForms;
use Filament\Forms\Contracts\HasForms;
use Filament\Forms\Form;
use Filament\Forms\Components\TextInput;
use Livewire\Component;

class ContactForm extends Component implements HasForms
{
    use InteractsWithForms;

    public ?array $data = [];

    public function mount(): void
    {
        $this->form->fill();
    }

    public function form(Form $form): Form
    {
        return $form
            ->schema([
                TextInput::make('name')->required(),
                TextInput::make('email')->email()->required(),
                Textarea::make('message')->rows(5)->required(),
            ])
            ->statePath('data');
    }

    public function submit(): void
    {
        $data = $this->form->getState();
        // ... do something with $data
    }

    public function render()
    {
        return view('livewire.contact-form');
    }
}

```

Blade:

```

<form wire:submit="submit">
    {{ $this->form }}
    <button type="submit">Send</button>
</form>

```

12. Gotchas & Sharp Edges

GOTCHA

A ``Repeater`` without ``->relationship()`` writes to a JSON column on the parent model. If your column is a string or you forget the ``array`` cast on the model, you'll see weird "must be array" errors on save. Either cast it or use ``->relationship()``.

GOTCHA

``->live()`` on every field is tempting, but it round-trips to the server on every keystroke. For large forms this gets sluggish. Use ``live(debounce: 500)`` or ``live(onBlur: true)`` unless you genuinely need per-keystroke reactivity.

WARNING

``FileUpload`` files in ``livewire-tmp/`` accumulate if users abandon forms. Configure Livewire's temp cleanup schedule (``php artisan livewire:configure-s3-upload-cleanup`` for S3, or scheduled cleanup of the local temp folder).

GOTCHA

``Select::make('user_id')->relationship('user', 'name')`` reads the ``user`` method on the model. The argument order is **relationship method name first, display column second** — not column name first. Easy mistake to make under deadline pressure.

NOTE

When using ``->unique()`` on a `TextInput` inside an `EditRecord` page, **always** pass ``ignoreRecord: true`` — otherwise the current record's existing value triggers the validation against itself and the form refuses to save.

GOTCHA

Conditional ``->required()`` returns true/false based on a closure, but conditional ``->visible()`` also affects validation: a hidden field is skipped entirely. If you want the field to be hidden but still validated, you need ``->disabled()`` + dehydrated state, not ``->hidden()``.

WARNING

``mutateFormDataBeforeCreate()`` and similar hooks live on the **Page** class (`CreatePost`, `EditPost`), not the Resource class. A common stumble: declaring them on the Resource and wondering why they never fire.

13. Cheatsheet — Field-to-Use-Case

Use case	Field
Name, title, single-line text	<code>TextInput</code>
Email, password, URL, phone	<code>TextInput</code> + <code>->email()</code> / <code>->password()</code> / <code>->url()</code> / <code>->tel()</code>
Long-form plain text	<code>Textarea</code>
Choose one from a list	<code>Select</code> or <code>Radio</code> or <code>ToggleButtons</code>
Choose many from a list	<code>Select::multiple()</code> or <code>CheckboxList</code>
On/off setting	<code>Toggle</code>
Acknowledge / consent	<code>Checkbox</code> + <code>->accepted()</code>
Date / datetime	<code>DatePicker</code> / <code>DateTimePicker</code>
Numeric range	<code>Slider</code>
Upload file(s)	<code>FileUpload</code>
Rich-text content	<code>RichEditor</code> or <code>MarkdownEditor</code>
Color choice	<code>ColorPicker</code>
Tags / keywords	<code>TagsInput</code>
Code snippet	<code>CodeEditor</code>
Arbitrary key/value pairs	<code>KeyValue</code>
Hidden / programmatic	<code>Hidden</code>
Read-only display	<code>Placeholder</code>
Repeating sub-records	<code>Repeater</code>
Heterogeneous content blocks	<code>Builder</code>

Use case	Layout
Multi-column row	Grid
Bordered card with heading	Section
Lightweight group	Fieldset
Tabbed sub-sections	Tabs
Multi-step wizard	Wizard
Side-by-side panels	Split

14. Things to Verify

Filament moves fast. Before relying on a specific syntax in production, double-check against the docs for your installed version:

1. **Namespace stability.** Most components are still in `Filament\Forms\Components\*`, but a few have migrated to `Filament\Schemas\Components\*` in v4+. The form-specific imports (`TextInput`, `Select`, etc.) haven't moved.
2. **Form vs Schema parameter.** v3 uses `Form $form`. v4+ may use `Schema $schema` in some contexts. Both still work in v5 for Resources — Filament keeps the form alias.
3. **New v4+ components.** `Slider`, `CodeEditor`, `TableRepeater`, `ModalTableSelect` only exist on v4+. Falls back to alternatives on v3.
4. `jsAction()`. v4.5+ only. On older versions use a regular `Action` or an Alpine snippet.
5. `saved(false)`. v4.5+ shortcut for the older `dehydrated(false)` — both still work.

End of notes.